

C++
11

constexpr 构造函数可以声明成=default (参见 7.1.4 节, 第 237 页) 的形式 (或者是删除函数的形式, 我们将在 13.1.6 节 (第 449 页) 介绍相关知识)。否则, constexpr 构造函数就必须既符合构造函数的要求 (意味着不能包含返回语句), 又符合 constexpr 函数的要求 (意味着它能拥有的唯一可执行语句就是返回语句 (参见 6.5.2 节, 第 214 页))。综合这两点可知, constexpr 构造函数体一般来说应该是空的。我们通过前置关键字 constexpr 就可以声明一个 constexpr 构造函数了:

```
class Debug {
public:
    constexpr Debug(bool b = true): hw(b), io(b), other(b) { }
    constexpr Debug(bool h, bool i, bool o):
        hw(h), io(i), other(o) { }
    constexpr bool any() { return hw || io || other; }
    void set_io(bool b) { io = b; }
    void set_hw(bool b) { hw = b; }
    void set_other(bool b) { other = b; }
private:
    bool hw;                // 硬件错误, 而非 IO 错误
    bool io;                // IO 错误
    bool other;             // 其他错误
};
```

300

constexpr 构造函数必须初始化所有数据成员, 初始值或者使用 constexpr 构造函数, 或者是一条常量表达式。

constexpr 构造函数用于生成 constexpr 对象以及 constexpr 函数的参数或返回类型:

```
constexpr Debug io_sub(false, true, false);    // 调试 IO
if (io_sub.any())                             // 等价于 if(true)
    cerr << "print appropriate error messages" << endl;
constexpr Debug prod(false);                  // 无调试
if (prod.any())                               // 等价于 if(false)
    cerr << "print an error message" << endl;
```

7.5.6 节练习

练习 7.53: 定义你自己的 Debug。

练习 7.54: Debug 中以 set_ 开头的成员应该被声明成 constexpr 吗? 如果不, 为什么?

练习 7.55: 7.5.5 节 (第 266 页) 的 Data 类是字面值常量类吗? 请解释原因。

7.6 类的静态成员

有的时候类需要它的一些成员与类本身直接相关, 而不是与类的各个对象保持关联。例如, 一个银行账户类可能需要一个数据成员来表示当前的基准利率。在此例中, 我们希望利率与类关联, 而非与类的每个对象关联。从实现效率的角度来看, 没必要每个对象都存储利率信息。而且更加重要的是, 一旦利率浮动, 我们希望所有的对象都能使用新值。

声明静态成员

我们通过在成员的声明之前加上关键字 `static` 使得其与类关联在一起。和其他成员一样，静态成员可以是 `public` 的或 `private` 的。静态数据成员的类型可以是常量、引用、指针、类类型等。

举个例子，我们定义一个类，用它表示银行的账户记录：

301

```
class Account {
public:
    void calculate() { amount += amount * interestRate; }
    static double rate() { return interestRate; }
    static void rate(double);
private:
    std::string owner;
    double amount;
    static double interestRate;
    static double initRate();
};
```

类的静态成员存在于任何对象之外，对象中不包含任何与静态数据成员有关的数据。因此，每个 `Account` 对象将包含两个数据成员：`owner` 和 `amount`。只存在一个 `interestRate` 对象而且它被所有 `Account` 对象共享。

类似的，静态成员函数也不与任何对象绑定在一起，它们不包含 `this` 指针。作为结果，静态成员函数不能声明成 `const` 的，而且我们也不能在 `static` 函数体内使用 `this` 指针。这一限制既适用于 `this` 的显式使用，也对调用非静态成员的隐式使用有效。

使用类的静态成员

我们使用作用域运算符直接访问静态成员：

```
double r;
r = Account::rate();           // 使用作用域运算符访问静态成员
```

虽然静态成员不属于类的某个对象，但是我们仍然可以使用类的对象、引用或者指针来访问静态成员：

```
Account ac1;
Account *ac2 = &ac1;
// 调用静态成员函数 rate 的等价形式
r = ac1.rate();               // 通过 Account 的对象或引用
r = ac2->rate();               // 通过指向 Account 对象的指针
```

成员函数不用通过作用域运算符就能直接使用静态成员：

```
class Account {
public:
    void calculate() { amount += amount * interestRate; }
private:
    static double interestRate;
    // 其他成员与之前的版本一致
};
```

302 定义静态成员

和其他的成员函数一样，我们既可以在类的内部也可以在类的外部定义静态成员函数。当在类的外部定义静态成员时，不能重复 `static` 关键字，该关键字只出现在类内部的声明语句：

```
void Account::rate(double newRate)
{
    interestRate = newRate;
}
```



和类的所有成员一样，当我们指向类外部的静态成员时，必须指明成员所属的类名。`static` 关键字则只出现在类内部的声明语句中。

因为静态数据成员不属于类的任何一个对象，所以它们并不是在创建类的对象时被定义的。这意味着它们不是由类的构造函数初始化的。而且一般来说，我们不能在类的内部初始化静态成员。相反的，必须在类的外部定义和初始化每个静态成员。和其他对象一样，一个静态数据成员只能定义一次。

类似于全局变量（参见 6.1.1 节，第 184 页），静态数据成员定义在任何函数之外。因此一旦它被定义，就将一直存在于程序的整个生命周期中。

我们定义静态数据成员的方式和在类的外部定义成员函数差不多。我们需要指定对象的类型名，然后是类名、作用域运算符以及成员自己的名字：

```
// 定义并初始化一个静态成员
double Account::interestRate = initRate();
```

这条语句定义了名为 `interestRate` 的对象，该对象是类 `Account` 的静态成员，其类型是 `double`。从类名开始，这条定义语句的剩余部分就都位于类的作用域之内了。因此，我们可以直接使用 `initRate` 函数。注意，虽然 `initRate` 是私有的，我们也能用它初始化 `interestRate`。和其他成员的定义一样，`interestRate` 的定义也可以访问类的私有成员。



要想确保对象只定义一次，最好的办法是把静态数据成员的定义与其他非内联函数的定义放在同一个文件中。

静态成员的内初始化

通常情况下，类的静态成员不应该在类的内部初始化。然而，我们可以为静态成员提供 `const` 整数类型的类内初始值，不过要求静态成员必须是字面值常量类型的 `constexpr`（参见 7.5.6 节，第 267 页）。初始值必须是常量表达式，因为这些成员本身就是常量表达式，所以它们能用在所有适合于常量表达式的地方。例如，我们可以用一个初始化了的静态数据成员指定数组成员的维度：

```
class Account {
public:
    static double rate() { return interestRate; }
    static void rate(double);
private:
```

```
static constexpr int period = 30;    // period 是常量表达式
double daily_tbl[period];
};
```

如果某个静态成员的应用场景仅限于编译器可以替换它的值的情况，则一个初始化的 `const` 或 `constexpr static` 不需要分别定义。相反，如果我们将它用于值不能替换的场景中，则该成员必须有一条定义语句。

例如，如果 `period` 的唯一用途就是定义 `daily_tbl` 的维度，则不需要在 `Account` 外面专门定义 `period`。此时，如果我们忽略了这条定义，那么对程序非常微小的改动也可能造成编译错误，因为程序找不到该成员的定义语句。举个例子，当需要把 `Account::period` 传递给一个接受 `const int&` 的函数时，必须定义 `period`。

如果在类的内部提供了一个初始值，则成员的定义不能再指定一个初始值了：

```
// 一个不带初始值的静态成员的定义
constexpr int Account::period;           // 初始值在类的定义内提供
```

Best
Practices

即使一个常量静态数据成员在类内部被初始化了，通常情况下也应该在类的外部定义一下该成员。

静态成员能用于某些场景，而普通成员不能

如我们所见，静态成员独立于任何对象。因此，在某些非静态数据成员可能非法的场合，静态成员却可以正常地使用。举个例子，静态数据成员可以是不完全类型（参见 7.3.3 节，第 249 页）。特别的，静态数据成员的类型可以就是它所属的类类型。而非静态数据成员则受到限制，只能声明成它所属类的指针或引用：

```
class Bar {
public:
    // ...
private:
    static Bar mem1;           // 正确：静态成员可以是不完全类型
    Bar *mem2;                 // 正确：指针成员可以是不完全类型
    Bar mem3;                  // 错误：数据成员必须是完全类型
};
```

静态成员和普通成员的另外一个区别是我们可以使用静态成员作为默认实参（参见 6.5.1 节，第 211 页）： 304

```
class Screen {
public:
    // bkground 表示一个在类中稍后定义的静态成员
    Screen& clear(char = bkground);
private:
    static const char bkground;
};
```

非静态数据成员不能作为默认实参，因为它的值本身属于对象的一部分，这么做的结果是无法真正提供一个对象以便从中获取成员的值，最终将引发错误。

7.6 节练习

练习 7.56: 什么是类的静态成员? 它有何优点? 静态成员与普通成员有何区别?

练习 7.57: 编写你自己的 Account 类。

练习 7.58: 下面的静态数据成员的声明和定义有错误吗? 请解释原因。

```
// example.h
class Example {
public:
    static double rate = 6.5;
    static const int vecSize = 20;
    static vector<double> vec(vecSize);
};
// example.C
#include "example.h"
double Example::rate;
vector<double> Example::vec;
```

小结

305

类是 C++ 语言中最基本的特性。类允许我们为自己的应用定义新类型，从而使得程序更加简洁且易于修改。

类有两项基本能力：一是数据抽象，即定义数据成员和函数成员的能力；二是封装，即保护类的成员不被随意访问的能力。通过将类的实现细节设为 `private`，我们就能完成类的封装。类可以将其他类或者函数设为友元，这样它们就能访问类的非公有成员了。

类可以定义一种特殊的成员函数：构造函数，其作用是控制初始化对象的方式。构造函数可以重载，构造函数应该使用构造函数初始值列表来初始化所有数据成员。

类还能定义可变或者静态成员。一个可变成员永远都不会是 `const`，即使在 `const` 成员函数内也能修改它的值；一个静态成员可以是函数也可以是数据，静态成员存在于所有对象之外。

术语表

抽象数据类型 (abstract data type) 封装 (隐藏) 了实现细节的数据结构。

访问说明符 (access specifier) 包括关键字 `public` 和 `private`。用于定义成员对类的用户可见还是只对类的友元和成员可见。在类中说明符可以出现多次，每个说明符的有效范围从它自身开始，到下一个说明符为止。

聚合类 (aggregate class) 只含有公有成员的类，并且没有类内初始值或者构造函数。聚合类的成员可以用花括号括起来的初始值列表进行初始化。

类 (class) C++ 提供的自定义数据类型的机制。类可以包含数据、函数和类型成员。一个类定义一种新的类型和一个新的作用域。

类的声明 (class declaration) 首先是关键字 `class` (或者 `struct`)，随后是类名以及分号。如果类已经声明而尚未定义，则它是一个不完全类型。

class 关键字 (class keyword) 用于定义类的关键字，默认情况下成员是 `private` 的。

类的作用域 (class scope) 每个类定义一个作用域。类作用域比其他作用域更加复

杂，类中定义的成员函数甚至有可能使用定义语句之后的名字。

常量成员函数 (const member function) 一个成员函数，在其中不能修改对象的普通 (即既不是 `static` 也不是 `mutable`) 数据成员。`const` 成员的 `this` 指针是指向常量的指针，通过区分函数是否是 `const` 可以进行重载。

构造函数 (constructor) 用于初始化对象的一种特殊的成员函数。构造函数应该给每个数据成员都赋一个合适的初始值。

构造函数初始值列表 (constructor initializer list) 说明一个类的数据成员的初始值，在构造函数体执行之前首先用初始值列表中的值初始化数据成员。未经初始值列表初始化的成员将被默认初始化。

转换构造函数 (converting constructor) 可以用一个实参调用的非显式构造函数。这样的函数隐式地将参数类型转换成类类型。

数据抽象 (data abstraction) 着重关注类型接口的一种编程技术。数据抽象令程序员可以忽略类型的实现细节，只关注类型执行的操作即可。数据抽象是面向对象编程和泛型编程的基础。

306